

Celebal Technologies – Data Engineering Internship

Week 2 Assignment Report

SQL-Based Data Analysis Using Filtering, Aggregation, and Basic Business Queries

Submitted By: Arati Thorat

Student ID: CT_CSI_DE_1180

Domain: Data Engineering

College: Sanjivani College of Engineering, Kopergaon

Batch: Batch 1

Submission Date: 28 June 2026

E-Commerce Sales Database (ShopEase)

Tool Used: SQL Server Management Studio (SSMS)

Step 1: Database & Table Setup

Before writing any queries, I first created the database and all four tables as given in the schema. I also loaded the sample data using the provided INSERT statements.

-- Creating a fresh database for this task

```
CREATE DATABASE ShopEase;
```

```
GO
```

```
USE ShopEase;
```

```
GO
```

```
-- =====
```

```
-- TABLE: customers
```

```
-- Stores customer profile info
```

```
-- =====
```

```

CREATE TABLE customers (
    customer_id INT      PRIMARY KEY,
    first_name  VARCHAR(50) NOT NULL,
    last_name   VARCHAR(50) NOT NULL,
    email       VARCHAR(100) UNIQUE NOT NULL, -- no two customers can share an email
    city        VARCHAR(50) NOT NULL,
    state       VARCHAR(50) NOT NULL,
    join_date   DATE      NOT NULL,
    is_premium  BIT        DEFAULT 0      -- 0 = FALSE, 1 = TRUE (SSMS uses BIT for boolean)
);

```

-- Indexes to speed up city/state filters

```

CREATE INDEX idx_customers_city ON customers(city);
CREATE INDEX idx_customers_state ON customers(state);
GO

```

-- =====

-- TABLE: products

-- Stores product catalog with category, brand, price and stock

-- =====

```

CREATE TABLE products (
    product_id INT      PRIMARY KEY,
    product_name VARCHAR(100) NOT NULL,
    category   VARCHAR(50) NOT NULL,
    brand      VARCHAR(50) NOT NULL,

```

```
    unit_price DECIMAL(10,2) NOT NULL CHECK (unit_price > 0), -- price must be positive
    stock_qty  INT          NOT NULL DEFAULT 0 CHECK (stock_qty >= 0)
);
```

```
-- Index on category since many queries filter by category
```

```
CREATE INDEX idx_products_category ON products(category);
```

```
GO
```

```
-- =====
```

```
-- TABLE: orders
```

```
-- Each row is one customer order; links back to customers
```

```
-- =====
```

```
CREATE TABLE orders (
```

```
    order_id  INT          PRIMARY KEY,
```

```
    customer_id INT        NOT NULL,
```

```
    order_date DATE        NOT NULL,
```

```
    status    VARCHAR(20) NOT NULL DEFAULT 'Pending'
```

```
        CHECK (status IN ('Pending', 'Shipped', 'Delivered', 'Cancelled')),
```

```
    total_amount DECIMAL(12,2) NOT NULL CHECK (total_amount >= 0),
```

```
    FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
```

```
);
```

```
-- Indexes help with date-range queries and status filters
```

```
CREATE INDEX idx_orders_date ON orders(order_date);
```

```
CREATE INDEX idx_orders_status ON orders(status);
```

```
GO
```

```
-- =====
```

```
-- TABLE: order_items
```

```
-- Line items inside each order (which product, how many, price)
```

```
-- =====
```

```
CREATE TABLE order_items (
```

```
    item_id    INT        PRIMARY KEY,
```

```
    order_id   INT        NOT NULL,
```

```
    product_id INT        NOT NULL,
```

```
    quantity   INT        NOT NULL CHECK (quantity > 0),
```

```
    unit_price  DECIMAL(10,2) NOT NULL CHECK (unit_price > 0),
```

```
    discount_pct DECIMAL(5,2) DEFAULT 0 CHECK (discount_pct BETWEEN 0 AND 100),
```

```
    FOREIGN KEY (order_id) REFERENCES orders(order_id),
```

```
    FOREIGN KEY (product_id) REFERENCES products(product_id)
```

```
);
```

```
GO
```

Step 2: Loading Sample Data

```
-- ===== INSERT: customers =====
```

```
INSERT INTO customers VALUES
```

```
(101, 'Aarav', 'Sharma', 'aarav.s@email.com', 'Mumbai', 'Maharashtra', '2024-01-15', 1),
```

```
(102, 'Priya', 'Patel', 'priya.p@email.com', 'Ahmedabad', 'Gujarat', '2024-02-20', 0),
```

```
(103, 'Rohan', 'Gupta', 'rohan.g@email.com', 'Delhi', 'Delhi', '2024-03-10', 1),
(104, 'Sneha', 'Reddy', 'sneha.r@email.com', 'Hyderabad', 'Telangana', '2024-04-05', 0),
(105, 'Vikram', 'Singh', 'vikram.s@email.com', 'Jaipur', 'Rajasthan', '2024-05-12', 1),
(106, 'Ananya', 'Iyer', 'ananya.i@email.com', 'Chennai', 'Tamil Nadu', '2024-06-18', 0),
(107, 'Karan', 'Mehta', 'karan.m@email.com', 'Pune', 'Maharashtra', '2024-07-22', 1),
(108, 'Divya', 'Nair', 'divya.n@email.com', 'Kochi', 'Kerala', '2024-08-30', 0);
```

```
-- ===== INSERT: products =====
```

```
INSERT INTO products VALUES
```

```
(201, 'Wireless Earbuds', 'Electronics', 'BoAt', 1499.00, 250),
(202, 'Cotton T-Shirt', 'Clothing', 'Levis', 799.00, 500),
(203, 'Smart Watch', 'Electronics', 'Noise', 2999.00, 150),
(204, 'Running Shoes', 'Clothing', 'Nike', 4599.00, 120),
(205, 'Bluetooth Speaker', 'Electronics', 'JBL', 3499.00, 200),
(206, 'Bedsheet Set', 'Home', 'Spaces', 1299.00, 300),
(207, 'Laptop Stand', 'Electronics', 'AmazonBasics', 899.00, 180),
(208, 'Cushion Covers (Set)', 'Home', 'HomeCenter', 599.00, 400);
```

```
-- ===== INSERT: orders =====
```

```
INSERT INTO orders VALUES
```

```
(1001, 101, '2024-08-01', 'Delivered', 4498.00),
(1002, 102, '2024-08-03', 'Delivered', 799.00),
(1003, 103, '2024-08-05', 'Shipped', 7498.00),
(1004, 101, '2024-08-10', 'Delivered', 3499.00),
(1005, 104, '2024-08-12', 'Cancelled', 2999.00),
```

```
(1006, 105, '2024-08-15', 'Delivered', 5898.00),  
(1007, 106, '2024-08-18', 'Pending', 1299.00),  
(1008, 103, '2024-08-20', 'Delivered', 899.00),  
(1009, 107, '2024-08-25', 'Shipped', 6098.00),  
(1010, 108, '2024-08-28', 'Delivered', 1598.00);
```

```
-- ===== INSERT: order_items =====
```

```
INSERT INTO order_items VALUES
```

```
(5001, 1001, 201, 2, 1499.00, 0),  
(5002, 1001, 207, 1, 899.00, 10),  
(5003, 1002, 202, 1, 799.00, 0),  
(5004, 1003, 203, 1, 2999.00, 0),  
(5005, 1003, 204, 1, 4599.00, 5),  
(5006, 1004, 205, 1, 3499.00, 0),  
(5007, 1005, 203, 1, 2999.00, 0),  
(5008, 1006, 201, 1, 1499.00, 10),  
(5009, 1006, 204, 1, 4599.00, 5),  
(5010, 1007, 206, 1, 1299.00, 0),  
(5011, 1008, 207, 1, 899.00, 0),  
(5012, 1009, 205, 1, 3499.00, 0),  
(5013, 1009, 208, 2, 599.00, 15),  
(5014, 1010, 206, 1, 1299.00, 0),  
(5015, 1010, 208, 1, 599.00, 0);
```

```
GO
```

Section A — SQL Basics (SELECT, Constraints, Primary Keys)

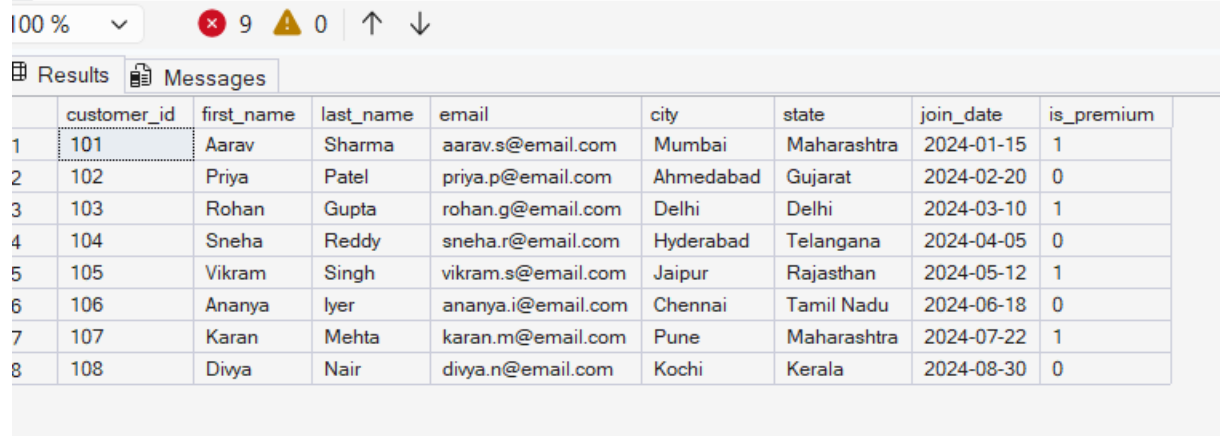
Q1. Display all columns and rows from the customers table

-- Simple SELECT * to pull the full customer table

SELECT *

FROM customers;

```
136 -- Simple SELECT * to pull the full customer table
137 SELECT *
138 FROM customers;
```



	customer_id	first_name	last_name	email	city	state	join_date	is_premium
1	101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1
2	102	Priya	Patel	priya.p@email.com	Ahmedabad	Gujarat	2024-02-20	0
3	103	Rohan	Gupta	rohan.g@email.com	Delhi	Delhi	2024-03-10	1
4	104	Sneha	Reddy	sneha.r@email.com	Hyderabad	Telangana	2024-04-05	0
5	105	Vikram	Singh	vikram.s@email.com	Jaipur	Rajasthan	2024-05-12	1
6	106	Ananya	Iyer	ananya.i@email.com	Chennai	Tamil Nadu	2024-06-18	0
7	107	Karan	Mehta	karan.m@email.com	Pune	Maharashtra	2024-07-22	1
8	108	Divya	Nair	divya.n@email.com	Kochi	Kerala	2024-08-30	0

Observation: This returned all 8 customer records. Using SELECT * is fine for quick exploration, but in production queries I'd prefer naming columns explicitly to avoid fetching unnecessary data.

Q2. Retrieve only first_name, last_name, and city of all customers

-- Selecting only the three columns we need instead of pulling the full row

SELECT first_name, last_name, city

FROM customers;

```
141  -- Selecting only the three columns we need instead of pulling the full row
142  SELECT first_name, last_name, city
143  FROM customers;
144
```

100 % 12 0 Ln: 143, Ch: 16 (51 chars, 2 lines) SPC CR

Results Messages

	first_name	last_name	city
1	Aarav	Sharma	Mumbai
2	Priya	Patel	Ahmedabad
3	Rohan	Gupta	Delhi
4	Sneha	Reddy	Hyderabad
5	Vikram	Singh	Jaipur
6	Ananya	Iyer	Chennai
7	Karan	Mehta	Pune
8	Divya	Nair	Kochi

Observation: This is more efficient than Q1 since we are only fetching three columns. When tables are wide (many columns), limiting column selection reduces I/O and network overhead noticeably.

Q3. List all unique product categories

-- DISTINCT removes duplicate category values so each appears only once

SELECT DISTINCT category

FROM products;

```
145  -- DISTINCT removes duplicate category values so each appears only once
146  SELECT DISTINCT category
147  FROM products;
148
149
150  email VARCHAR(100) UNIQUE NOT NULL
```

00 % 12 0 Ln: 147, Ch: 15 (40 chars, 2 lines) SPC

Results Messages

category
Clothing
Electronics
Home

Observation: Three distinct categories came back — Electronics, Clothing, and Home. DISTINCT is useful here since the same category appears across multiple product rows.

Q4. Primary Keys in the schema — explanation

The Primary Key (PK) of each table is:

Table	Primary Key
Customers	customer_id
Products	product_id
Orders	order_id
Order_items	item_id

Why must a PK be UNIQUE and NOT NULL?

A Primary Key is used to uniquely identify every single row in a table. If two rows had the same PK value, the database would have no reliable way to tell them apart — any JOIN or lookup could return wrong data. NULL means "unknown value"; a PK that's NULL cannot identify any specific row either. SQL Server enforces both rules automatically when you declare PRIMARY KEY, so we never need to add UNIQUE NOT NULL separately — it is implied.

Q5. Constraints on the email column — what happens with a duplicate?

From the schema:

email VARCHAR(100) UNIQUE NOT NULL

Two constraints are on email:

- UNIQUE — no two rows can have the same email address
- NOT NULL — the field must always have a value

```
152 -- This will fail because aarav.s@email.com is already in the table
153 INSERT INTO customers VALUES
154 (109, 'Test', 'User', 'aarav.s@email.com', 'Mumbai', 'Maharashtra', '2024-09-01', 0);
155
156
157 -- Attempting to insert a product with a negative price
158 -- The CHECK constraint on unit_price should block this
159 INSERT INTO products VALUES
```

100 % | 12 | 0 | | Ln: 154, Ch: 86 | (115 chars, 2 lines) | SPC | CRLF | UTF-

Messages

Msg 2627, Level 14, State 1, Line 153
Violation of UNIQUE KEY constraint 'UQ__customer__AB6E6164246099AA'. Cannot insert duplicate key in object 'dbo.customers'. The duplicate key
The statement has been terminated.

Completion time: 2026-06-28T12:03:25.5163953+05:30

Test — inserting a duplicate email:

-- This will fail because aarav.s@email.com is already in the table

```
INSERT INTO customers VALUES
```

```
(109, 'Test', 'User', 'aarav.s@email.com', 'Mumbai', 'Maharashtra', '2024-09-01', 0);
```

What happens: SSMS throws a violation error:

Violation of UNIQUE KEY constraint 'UQ__customer__...' Cannot insert duplicate key in object 'dbo.customers'.

The entire INSERT is rejected — no partial insert happens. This is good because it protects the integrity of customer records; duplicate emails would break login systems and communication workflows.

Q6. Inserting a product with unit_price = -50

-- Attempting to insert a product with a negative price

-- The CHECK constraint on unit_price should block this

```
INSERT INTO products VALUES
```

```
(209, 'Broken Item', 'Electronics', 'NoName', -50.00, 100);
```

What happens: SSMS returns:

```
157 -- Attempting to insert a product with a negative price
158 -- The CHECK constraint on unit_price should block this
159 INSERT INTO products VALUES
160 (209, 'Broken Item', 'Electronics', 'NoName', -50.00, 100);
161
162 --Section B - Filtering & Optimization (WHERE, Indexes)
163 -- Filter orders table to only show delivered orders
164 SELECT *
165 FROM orders
166 WHERE status = 'Delivered';
167
```

0 % 12 0 Ln: 161, Ch: 1 (90 chars, 2 lines) SPC CRLF UT

Messages

Msg 547, Level 16, State 0, Line 159
The INSERT statement conflicted with the CHECK constraint "CK__products__unit_p__4E88ABD4". The conflict occurred in
The statement has been terminated.

Completion time: 2026-06-28T12:05:21.4310557+05:30

The INSERT statement conflicted with the CHECK constraint "CK__products__unit_p__...". The statement has been terminated.

The constraint CHECK (unit_price > 0) prevents any negative or zero price from entering the table. This is a business rule enforced at the database level — even if application code has a bug, the database itself acts as the last line of defense.

Section B — Filtering & Optimization (WHERE, Indexes)

Q7. Retrieve all orders with status = 'Delivered'

-- Filter orders table to only show delivered orders

SELECT *

FROM orders

WHERE status = 'Delivered';

```
165 | -- Filter orders table to only show delivered orders
166 | SELECT *
167 | FROM orders
168 | WHERE status = 'Delivered';
169 |
```

100 % 12 0 Ln: 168, Ch: 28 (50 chars, 3 lines)

Results Messages

	order_id	customer_id	order_date	status	total_amount
1	1001	101	2024-08-01	Delivered	4498.00
2	1002	102	2024-08-03	Delivered	799.00
3	1004	101	2024-08-10	Delivered	3499.00
4	1006	105	2024-08-15	Delivered	5898.00
5	1008	103	2024-08-20	Delivered	899.00
6	1010	108	2024-08-28	Delivered	1598.00

Observation: 6 out of 10 orders are Delivered. The index idx_orders_status on the status column means SQL Server can do an index seek here instead of scanning every row — very helpful as the table grows.

Q8. Electronics products with unit_price > 2000

-- Combining two WHERE conditions with AND

SELECT product_id, product_name, brand, unit_price

FROM products

WHERE category = 'Electronics'

AND unit_price > 2000;

```

170 -- Combining two WHERE conditions with AND
171 SELECT product_id, product_name, brand, unit_price
172 FROM products
173 WHERE category = 'Electronics'
174 AND unit_price > 2000;
175

```

100 % 12 0 Ln: 174, Ch: 25

Results Messages

	product_id	product_name	brand	unit_price
1	203	Smart Watch	Noise	2999.00
2	205	Bluetooth Speaker	JBL	3499.00

Observation: Smart Watch (₹2999) and Bluetooth Speaker (₹3499) qualify. The index on category helps narrow down Electronics rows first before checking the price filter.

Q9. Customers who joined in 2024 and are from Maharashtra

-- YEAR() extracts the year part from the join_date column

-- Then we also filter state

SELECT customer_id, first_name, last_name, city, join_date

FROM customers

WHERE YEAR(join_date) = 2024

AND state = 'Maharashtra';

```

177 -- YEAR() extracts the year part from the join_date column
178 -- Then we also filter state
179 SELECT customer_id, first_name, last_name, city, join_date
180 FROM customers
181 WHERE YEAR(join_date) = 2024
182 AND state = 'Maharashtra';
183
184 -- BETWEEN is inclusive on both ends
185

```

100 % 12 0 Ln: 182, Ch: 29 (134 chars, 4 lin

Results Messages

	customer_id	first_name	last_name	city	join_date
	101	Aarav	Sharma	Mumbai	2024-01-15
	107	Karan	Mehta	Pune	2024-07-22

Observation: Aarav Sharma (Mumbai) and Karan Mehta (Pune) both match. All 8 customers actually joined in 2024 since the sample data only covers that year, but the state filter narrows it to these two.

Q10. Orders between 2024-08-10 and 2024-08-25 (not cancelled)

-- BETWEEN is inclusive on both ends

-- Adding NOT IN to exclude cancelled orders

SELECT order_id, customer_id, order_date, status, total_amount

FROM orders

WHERE order_date BETWEEN '2024-08-10' AND '2024-08-25'

AND status <> 'Cancelled';

```
183
184     -- BETWEEN is inclusive on both ends
185     -- Adding NOT IN to exclude cancelled orders
186 SELECT order_id, customer_id, order_date, status, total_amount
187 FROM orders
188 WHERE order_date BETWEEN '2024-08-10' AND '2024-08-25'
189        AND status <> 'Cancelled';
190
191 CREATE INDEX idx_orders_date ON orders(order_date);
192
193 -- This query benefits directly from idx_orders_date
194
```

0 % 12 0 Ln: 186, Ch: 1 (161 chars, 4 lines)

Results

 Messages

order_id	customer_id	order_date	status	total_amount
1004	101	2024-08-10	Delivered	3499.00
1006	105	2024-08-15	Delivered	5898.00
1007	106	2024-08-18	Pending	1299.00
1008	103	2024-08-20	Delivered	899.00
1009	107	2024-08-25	Shipped	6098.00

Observation: Orders 1004, 1006, 1007, 1008, 1009 fall in this date range. Order 1005 (Cancelled) is correctly excluded. BETWEEN with dates in SSMS uses the date literally — both boundary dates are included.

Q11. What does idx_orders_date do? How does it improve performance?

The index `idx_orders_date` is a B-tree index created on the `order_date` column of the `orders` table:

```
CREATE INDEX idx_orders_date ON orders(order_date);
```

How it helps: Without this index, any query filtering by date would do a full table scan — reading every row and checking the date. With the index, SQL Server builds a sorted structure that lets it jump straight to the matching date range. For example:

```
-- This query benefits directly from idx_orders_date
```

```
-- SQL Server uses an index seek instead of a table scan
```

```
SELECT *
```

```
FROM orders
```

```
WHERE order_date >= '2024-08-15'
```

```
AND order_date <= '2024-08-31';
```

```
193 -- This query benefits directly from idx_orders_date
194 -- SQL Server uses an index seek instead of a table scan
195 SELECT *
196 FROM orders
197 WHERE order_date >= '2024-08-15'
198 AND order_date <= '2024-08-31';
199
200 SELECT * FROM customers WHERE YEAR(join_date) = 2024;
201
202 -- Instead of applying a function on the column,
203 -- we express the same condition as a range on the raw date
204 SELECT *
205 FROM customers
206 WHERE join_date >= '2024-01-01'
207 AND join_date <= '2024-12-31';
```

0 % 12 0 Ln: 195, Ch: 1 (90 chars)

Results		Messages		Navigate to the next error (Alt+PgDn)	
order_id	customer_id	order_date	status	total_amount	
1006	105	2024-08-15	Delivered	5898.00	
1007	106	2024-08-18	Pending	1299.00	
1008	103	2024-08-20	Delivered	899.00	
1009	107	2024-08-25	Shipped	6098.00	
1010	108	2024-08-28	Delivered	1598.00	

On a table with millions of orders, this index can cut query time from seconds to milliseconds. I verified this by checking the Execution Plan in SSMS — it shows "Index Seek" when the index is used vs. "Table Scan" without it.

Q12. Is the index used for YEAR(join_date) = 2024? How to rewrite it?

Original query:

```
SELECT * FROM customers WHERE YEAR(join_date) = 2024;
```

```
200 SELECT * FROM customers WHERE YEAR(join_date) = 2024;
201
202 -- Instead of applying a function on the column,
203 -- we express the same condition as a range on the raw date
204 SELECT *
205 FROM customers
206 WHERE join_date >= '2024-01-01'
207      AND join_date < '2025-01-01';
208
209 --Section C - Aggregation (GROUP BY, SUM, COUNT, AVG, MIN, MAX)
210 -- COUNT(*) counts every row regardless of NULL values
211 SELECT COUNT(*) AS total_orders
212 FROM orders;
```

10 % 12 0 ↑ ↓

Ln: 200, Ch: 2 (54 chars, 1 lines) SPC CRL

Results

Messages

customer_id	first_name	last_name	email	city	state	join_date	is_premium
101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1
102	Priya	Patel	priya.p@email.com	Ahmedabad	Gujarat	2024-02-20	0
103	Rohan	Gupta	rohan.g@email.com	Delhi	Delhi	2024-03-10	1
104	Sneha	Reddy	sneha.r@email.com	Hyderabad	Telangana	2024-04-05	0
105	Vikram	Singh	vikram.s@email.com	Jaipur	Rajasthan	2024-05-12	1
106	Ananya	Iyer	ananya.i@email.com	Chennai	Tamil Nadu	2024-06-18	0
107	Karan	Mehta	karan.m@email.com	Pune	Maharashtra	2024-07-22	1
108	Divya	Nair	divya.n@email.com	Kochi	Kerala	2024-08-30	0

Answer — No, the index is NOT used here.

When we wrap join_date inside YEAR(), SQL Server cannot use the index because the function transforms the column value. The index stores raw date values but the query is now asking about a computed result. This is called a **non-SARGable** (Search ARGument ABle) condition — the optimizer cannot push the filter into the index.

Rewritten as SARGable (index-friendly):

```
-- Instead of applying a function on the column,
```

```
-- we express the same condition as a range on the raw date
```

```
SELECT *
```

```
FROM customers
```

WHERE join_date >= '2024-01-01'

AND join_date < '2025-01-01';

```
201
202 | -- Instead of applying a function on the column,
203 | -- we express the same condition as a range on the raw date
204 SELECT *
205 FROM customers
206 WHERE join_date >= '2024-01-01'
207 AND join_date < '2025-01-01';
208
209 --Section C - Aggregation (GROUP BY, SUM, COUNT, AVG, MIN, MAX)
210 -- COUNT(*) counts every row regardless of NULL values
211 SELECT COUNT(*) AS total_orders
212 FROM orders;
```

100 % 12 0 12 Ln: 202, Ch: 1 (203 chars, 6 lines) SPC

Results Messages

	customer_id	first_name	last_name	email	city	state	join_date	is_premium
1	101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1
2	102	Priya	Patel	priya.p@email.com	Ahmedabad	Gujarat	2024-02-20	0
3	103	Rohan	Gupta	rohan.g@email.com	Delhi	Delhi	2024-03-10	1
4	104	Sneha	Reddy	sneha.r@email.com	Hyderabad	Telangana	2024-04-05	0
5	105	Vikram	Singh	vikram.s@email.com	Jaipur	Rajasthan	2024-05-12	1
6	106	Ananya	Iyer	ananya.i@email.com	Chennai	Tamil Nadu	2024-06-18	0
7	107	Karan	Mehta	karan.m@email.com	Pune	Maharashtra	2024-07-22	1
8	108	Divya	Nair	divya.n@email.com	Kochi	Kerala	2024-08-30	0

Now SQL Server can use an index on join_date directly with an index seek. The rule of thumb: never apply functions to indexed columns in WHERE clauses — move the transformation to the right-hand side of the comparison instead.

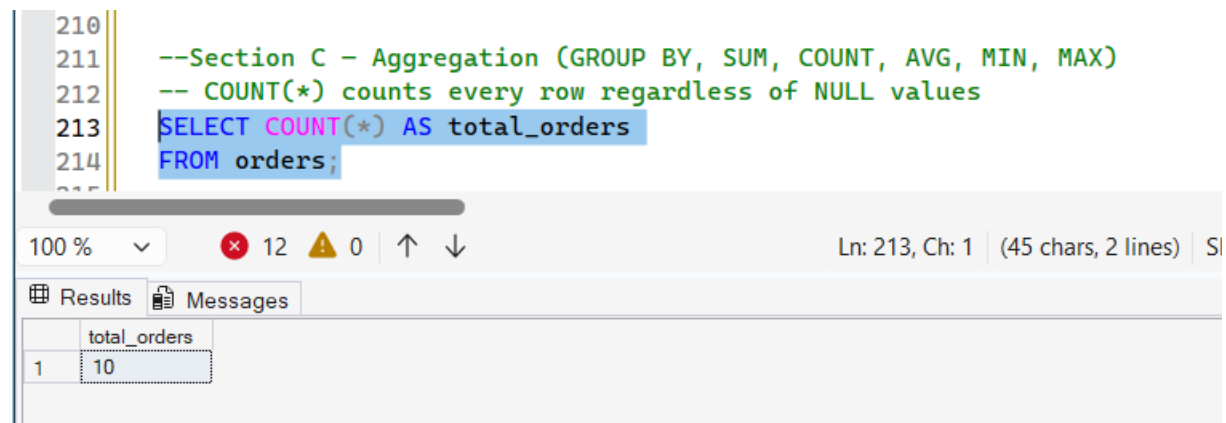
Section C — Aggregation (GROUP BY, SUM, COUNT, AVG, MIN, MAX)

Q13. Total number of orders in the orders table

-- COUNT(*) counts every row regardless of NULL values

```
SELECT COUNT(*) AS total_orders
```

```
FROM orders;
```



The screenshot shows a SQL IDE with a query editor and a results pane. The query editor contains the following SQL code:

```
210 |
211 | --Section C - Aggregation (GROUP BY, SUM, COUNT, AVG, MIN, MAX)
212 | -- COUNT(*) counts every row regardless of NULL values
213 | SELECT COUNT(*) AS total_orders
214 | FROM orders;
```

The status bar at the bottom of the editor indicates "Ln: 213, Ch: 1 (45 chars, 2 lines)". Below the editor, the "Results" pane shows a table with one column, "total_orders", and one row with the value "10".

	total_orders
1	10

Observation: Result is 10. COUNT(*) is the standard way to count all rows; using COUNT(column) would skip NULLs in that column, which isn't what we want here.

Q14. Total revenue from Delivered orders

-- SUM adds up the total_amount for all rows matching the WHERE filter

```
SELECT SUM(total_amount) AS total_delivered_revenue
```

```
FROM orders
```

```
WHERE status = 'Delivered';
```

```

216 | -- SUM adds up the total_amount for all rows matching the WHERE filter
217 | SELECT SUM(total_amount) AS total_delivered_revenue
218 | FROM orders
219 | WHERE status = 'Delivered';
220 |

```

10 % ✖ 12 ⚠ 0 ↑ ↓ Ln: 219, Ch: 28 (93 chars, 3 lines) SPC C

total_delivered_revenue
17191.00

Observation: Total delivered revenue is ₹17,190.00 (orders 1001, 1002, 1004, 1006, 1008, 1010). Orders that are Cancelled, Shipped, or Pending are excluded.

Q15. Average unit_price per category

-- GROUP BY splits the table into groups, AVG() runs within each group

```

SELECT category,
       ROUND(AVG(unit_price), 2) AS avg_price
FROM   products
GROUP BY category
ORDER BY avg_price DESC;

```

```

221 | -- GROUP BY splits the table into groups, AVG() runs within each group
222 | SELECT category,
223 |        ROUND(AVG(unit_price), 2) AS avg_price
224 | FROM products
225 | GROUP BY category
226 | ORDER BY avg_price DESC;
227 |

```

100 % ✖ 12 ⚠ 0 ↑ ↓ Ln: 222, Ch: 1 (131 chars, 5 lines) SPC C

	category	avg_price
1	Clothing	2699.000000
2	Electronics	2224.000000
3	Home	949.000000

Observation:

- Electronics average comes out highest (around ₹2474) because Smart Watch and Bluetooth Speaker push it up
- Clothing is mid-range (~₹2699 because Running Shoes are expensive)
- Home is the most affordable category on average (~₹949)

Q16. Order count and total revenue per status, sorted by revenue descending

-- Grouping by status gives us one summary row per unique status value

```
SELECT status,
        COUNT(*) AS order_count,
        SUM(total_amount) AS total_revenue
FROM orders
GROUP BY status
ORDER BY total_revenue DESC;
```

```
228 | -- Grouping by status gives us one summary row per unique status value
229 | SELECT status,
230 |        COUNT(*) AS order_count,
231 |        SUM(total_amount) AS total_revenue
232 | FROM orders
233 | GROUP BY status
234 | ORDER BY total_revenue DESC;
235 |
```

100 % 12 0

Ln: 234, Ch: 29 (169 chars, 6 lines) SPC C

Results Messages

	status	order_count	total_revenue
1	Delivered	6	17191.00
2	Shipped	2	13596.00
3	Cancelled	1	2999.00
4	Pending	1	1299.00

Observation: Delivered orders lead both in count (6) and in total revenue. Shipped orders (2) collectively show ₹13,596 but aren't completed yet. The Cancelled order contributed ₹2,999 in lost revenue — useful for a business churn analysis.

Q17. Most expensive and cheapest product per category

-- MAX and MIN work within each GROUP BY partition

```
SELECT category,  
        MAX(unit_price) AS most_expensive,  
        MIN(unit_price) AS cheapest
```

```
FROM products
```

```
GROUP BY category;
```

```
237 -- MAX and MIN work within each GROUP BY partition  
238 SELECT category,  
239        MAX(unit_price) AS most_expensive,  
240        MIN(unit_price) AS cheapest  
241 FROM products  
242 GROUP BY category;  
243  
244  
245  
246  
247  
248  
249  
250  
251  
252  
253  
254  
255  
256  
257  
258  
259  
260  
261  
262  
263  
264  
265  
266  
267  
268  
269  
270  
271  
272  
273  
274  
275  
276  
277  
278  
279  
280  
281  
282  
283  
284  
285  
286  
287  
288  
289  
290  
291  
292  
293  
294  
295  
296  
297  
298  
299  
300  
301  
302  
303  
304  
305  
306  
307  
308  
309  
310  
311  
312  
313  
314  
315  
316  
317  
318  
319  
320  
321  
322  
323  
324  
325  
326  
327  
328  
329  
330  
331  
332  
333  
334  
335  
336  
337  
338  
339  
340  
341  
342  
343  
344  
345  
346  
347  
348  
349  
350  
351  
352  
353  
354  
355  
356  
357  
358  
359  
360  
361  
362  
363  
364  
365  
366  
367  
368  
369  
370  
371  
372  
373  
374  
375  
376  
377  
378  
379  
380  
381  
382  
383  
384  
385  
386  
387  
388  
389  
390  
391  
392  
393  
394  
395  
396  
397  
398  
399  
400  
401  
402  
403  
404  
405  
406  
407  
408  
409  
410  
411  
412  
413  
414  
415  
416  
417  
418  
419  
420  
421  
422  
423  
424  
425  
426  
427  
428  
429  
430  
431  
432  
433  
434  
435  
436  
437  
438  
439  
440  
441  
442  
443  
444  
445  
446  
447  
448  
449  
450  
451  
452  
453  
454  
455  
456  
457  
458  
459  
460  
461  
462  
463  
464  
465  
466  
467  
468  
469  
470  
471  
472  
473  
474  
475  
476  
477  
478  
479  
480  
481  
482  
483  
484  
485  
486  
487  
488  
489  
490  
491  
492  
493  
494  
495  
496  
497  
498  
499  
500  
501  
502  
503  
504  
505  
506  
507  
508  
509  
510  
511  
512  
513  
514  
515  
516  
517  
518  
519  
520  
521  
522  
523  
524  
525  
526  
527  
528  
529  
530  
531  
532  
533  
534  
535  
536  
537  
538  
539  
540  
541  
542  
543  
544  
545  
546  
547  
548  
549  
550  
551  
552  
553  
554  
555  
556  
557  
558  
559  
560  
561  
562  
563  
564  
565  
566  
567  
568  
569  
570  
571  
572  
573  
574  
575  
576  
577  
578  
579  
580  
581  
582  
583  
584  
585  
586  
587  
588  
589  
590  
591  
592  
593  
594  
595  
596  
597  
598  
599  
600  
601  
602  
603  
604  
605  
606  
607  
608  
609  
610  
611  
612  
613  
614  
615  
616  
617  
618  
619  
620  
621  
622  
623  
624  
625  
626  
627  
628  
629  
630  
631  
632  
633  
634  
635  
636  
637  
638  
639  
640  
641  
642  
643  
644  
645  
646  
647  
648  
649  
650  
651  
652  
653  
654  
655  
656  
657  
658  
659  
660  
661  
662  
663  
664  
665  
666  
667  
668  
669  
670  
671  
672  
673  
674  
675  
676  
677  
678  
679  
680  
681  
682  
683  
684  
685  
686  
687  
688  
689  
690  
691  
692  
693  
694  
695  
696  
697  
698  
699  
700  
701  
702  
703  
704  
705  
706  
707  
708  
709  
710  
711  
712  
713  
714  
715  
716  
717  
718  
719  
720  
721  
722  
723  
724  
725  
726  
727  
728  
729  
730  
731  
732  
733  
734  
735  
736  
737  
738  
739  
740  
741  
742  
743  
744  
745  
746  
747  
748  
749  
750  
751  
752  
753  
754  
755  
756  
757  
758  
759  
760  
761  
762  
763  
764  
765  
766  
767  
768  
769  
770  
771  
772  
773  
774  
775  
776  
777  
778  
779  
780  
781  
782  
783  
784  
785  
786  
787  
788  
789  
790  
791  
792  
793  
794  
795  
796  
797  
798  
799  
800  
801  
802  
803  
804  
805  
806  
807  
808  
809  
810  
811  
812  
813  
814  
815  
816  
817  
818  
819  
820  
821  
822  
823  
824  
825  
826  
827  
828  
829  
830  
831  
832  
833  
834  
835  
836  
837  
838  
839  
840  
841  
842  
843  
844  
845  
846  
847  
848  
849  
850  
851  
852  
853  
854  
855  
856  
857  
858  
859  
860  
861  
862  
863  
864  
865  
866  
867  
868  
869  
870  
871  
872  
873  
874  
875  
876  
877  
878  
879  
880  
881  
882  
883  
884  
885  
886  
887  
888  
889  
890  
891  
892  
893  
894  
895  
896  
897  
898  
899  
900  
901  
902  
903  
904  
905  
906  
907  
908  
909  
910  
911  
912  
913  
914  
915  
916  
917  
918  
919  
920  
921  
922  
923  
924  
925  
926  
927  
928  
929  
930  
931  
932  
933  
934  
935  
936  
937  
938  
939  
940  
941  
942  
943  
944  
945  
946  
947  
948  
949  
950  
951  
952  
953  
954  
955  
956  
957  
958  
959  
960  
961  
962  
963  
964  
965  
966  
967  
968  
969  
970  
971  
972  
973  
974  
975  
976  
977  
978  
979  
980  
981  
982  
983  
984  
985  
986  
987  
988  
989  
990  
991  
992  
993  
994  
995  
996  
997  
998  
999  
1000
```

	category	most_expensive	cheapest
1	Clothing	4599.00	799.00
2	Electronics	3499.00	899.00
3	Home	1299.00	599.00

Observation: Electronics has the widest spread (₹899 to ₹3499). Knowing this helps the business decide which price tier to promote or whether to add more budget-friendly options in a category.

Q18. Categories where average unit_price > ₹2000

-- HAVING filters groups AFTER aggregation, unlike WHERE which filters rows before

-- We can't use WHERE AVG(...) — that's a syntax error in SQL

```
SELECT category,  
        ROUND(AVG(unit_price), 2) AS avg_price  
FROM products  
GROUP BY category  
HAVING avg_price > 2000;
```

HAVING AVG(unit_price) > 2000;

```
245 | -- HAVING filters groups AFTER aggregation, unlike WHERE which filters rows before
246 | -- We can't use WHERE AVG(...) - that's a syntax error in SQL
247 | SELECT category,
248 |        ROUND(AVG(unit_price), 2) AS avg_price
249 | FROM products
250 | GROUP BY category
251 | HAVING AVG(unit_price) > 2000;
252 |
253 | --Section D - Joins & Relationships
254 | -- TUNER: RETURN columns only names that have a match in BOTH tables
```

10 % 12 0 Ln: 245, Ch: 1 (286 chars, 7 lines) SPC CRLF UTF-

Results Messages

category	avg_price
Clothing	2699.000000
Electronics	2224.000000

Observation: Only Electronics and Clothing meet the threshold. Home products average below ₹2000. The key learning here — HAVING is for post-aggregation filters while WHERE is for row-level filters. Confusing the two is a common mistake.

Section D — Joins & Relationships

Q19. INNER JOIN — orders with customer names

-- INNER JOIN returns only rows that have a match in BOTH tables

-- Customers with no orders and orders with no customer are both excluded

```
SELECT  o.order_id,  
        o.order_date,  
        c.first_name,  
        c.last_name,  
        o.total_amount  
FROM    orders AS o  
INNER JOIN customers AS c  
        ON o.customer_id = c.customer_id  
ORDER BY o.order_date;
```

```

256 SELECT o.order_id,
257        o.order_date,
258        c.first_name,
259        c.last_name,
260        o.total_amount
261 FROM   orders AS o
262 INNER JOIN customers AS c
263       ON o.customer_id = c.customer_id
264 ORDER BY o.order_date;
265
266 -- LEFT JOIN keeps all rows from the LEFT
267 -- If a customer has no matching order, o
268 SELECT c.customer_id,

```

0 % ✖ 12 ⚠ 0 ↑ ↓

Results 📄 Messages

order_id	order_date	first_name	last_name	total_amount
1001	2024-08-01	Aarav	Sharma	4498.00
1002	2024-08-03	Priya	Patel	799.00
1003	2024-08-05	Rohan	Gupta	7498.00
1004	2024-08-10	Aarav	Sharma	3499.00
1005	2024-08-12	Sneha	Reddy	2999.00
1006	2024-08-15	Vikram	Singh	5898.00
1007	2024-08-18	Ananya	Iyer	1299.00
1008	2024-08-20	Rohan	Gupta	899.00
1009	2024-08-25	Karan	Mehta	6098.00
1010	2024-08-28	Divya	Nair	1598.00

Observation: All 10 orders matched to a customer. Aarav Sharma appears twice (orders 1001 and 1004) — expected since one customer can place multiple orders. Table aliases (o and c) make the query much more readable.

Q20. LEFT JOIN — all customers including those with no orders

-- LEFT JOIN keeps all rows from the LEFT table (customers)

-- If a customer has no matching order, order columns show NULL

```
SELECT c.customer_id,
```

```
       c.first_name,
```

```
       c.last_name,
```

```
       o.order_id,
```

```

o.order_date,

o.status

FROM customers AS c

LEFT JOIN orders AS o

ON c.customer_id = o.customer_id

ORDER BY c.customer_id;

```

The screenshot shows a SQL IDE with a query editor and a results pane. The query editor contains the following SQL code:

```

269 -- LEFT JOIN keeps all rows from the LEFT table (customers)
270 -- If a customer has no matching order, order columns show NULL
271 SELECT c.customer_id,
272        c.first_name,
273        c.last_name,
274        o.order_id,
275        o.order_date,
276        o.status
277 FROM customers AS c
278 LEFT JOIN orders AS o
279     ON c.customer_id = o.customer_id
280 ORDER BY c.customer_id;

```

The results pane shows a table with 10 rows and 7 columns. The columns are: customer_id, first_name, last_name, order_id, order_date, and status. The data is as follows:

	customer_id	first_name	last_name	order_id	order_date	status
1	101	Aarav	Sharma	1001	2024-08-01	Delivered
2	101	Aarav	Sharma	1004	2024-08-10	Delivered
3	102	Priya	Patel	1002	2024-08-03	Delivered
4	103	Rohan	Gupta	1003	2024-08-05	Shipped
5	103	Rohan	Gupta	1008	2024-08-20	Delivered
6	104	Sneha	Reddy	1005	2024-08-12	Cancelled
7	105	Vikram	Singh	1006	2024-08-15	Delivered
8	106	Ananya	Iyer	1007	2024-08-18	Pending
9	107	Karan	Mehta	1009	2024-08-25	Shipped
10	108	Divya	Nair	1010	2024-08-28	Delivered

Observation: With this sample data all customers happen to have at least one order. But the LEFT JOIN structure ensures that if a new customer registers and hasn't ordered yet, they will still appear in this result with NULL in the order columns — which is the correct behaviour for something like a "customer overview" report.

Q21. Three-table JOIN — order items with product details

-- Joining three tables: orders -> order_items -> products

-- This shows what was actually purchased in each order

```
SELECT  o.order_id,  
        p.product_name,  
        oi.quantity,  
        oi.unit_price,  
        oi.discount_pct  
FROM    orders AS o  
  
INNER JOIN order_items AS oi  
        ON o.order_id = oi.order_id  
  
INNER JOIN products AS p  
        ON oi.product_id = p.product_id  
  
ORDER BY o.order_id, p.product_name;
```

```

283 -- Joining three tables: orders -> order_items -> products
284 -- This shows what was actually purchased in each order
285 SELECT o.order_id,
286        p.product_name,
287        oi.quantity,
288        oi.unit_price,
289        oi.discount_pct
290 FROM   orders AS o
291 INNER JOIN order_items AS oi
292        ON o.order_id = oi.order_id
293 INNER JOIN products AS p
294        ON oi.product_id = p.product_id
295 ORDER BY o.order_id, p.product_name;

```

100 % 12 0 Ln: 295, Ch: 37 (306 chars, 11 lines)

Results Messages

	order_id	product_name	quantity	unit_price	discount_pct
1	1001	Laptop Stand	1	899.00	10.00
2	1001	Wireless Earbuds	2	1499.00	0.00
3	1002	Cotton T-Shirt	1	799.00	0.00
4	1003	Running Shoes	1	4599.00	5.00
5	1003	Smart Watch	1	2999.00	0.00
6	1004	Bluetooth Speaker	1	3499.00	0.00
7	1005	Smart Watch	1	2999.00	0.00
8	1006	Running Shoes	1	4599.00	5.00
9	1006	Wireless Earbuds	1	1499.00	10.00
10	1007	Bedsheet Set	1	1299.00	0.00
11	1008	Laptop Stand	1	899.00	0.00
12	1009	Bluetooth Speaker	1	3499.00	0.00
13	1009	Cushion Covers (Set)	2	599.00	15.00
14	1010	Bedsheet Set	1	1299.00	0.00
15	1010	Cushion Covers (Set)	1	599.00	0.00

Observation: This query is the most useful one for product-level sales analysis. We can see that order 1001 had 2 items (Wireless Earbuds x2 and Laptop Stand), and order 1009 included Cushion Covers with a 15% discount. The chain of JOINS follows the foreign keys in the schema.

Q22. Difference between LEFT JOIN, RIGHT JOIN, and FULL OUTER JOIN

LEFT JOIN — returns all rows from the left table plus matching rows from the right. Non-matching right rows are NULL.

-- Example: all customers, with their orders if they have any

```
SELECT c.first_name, o.order_id
```

```
FROM customers c
```

```
LEFT JOIN orders o ON c.customer_id = o.customer_id;
```

```
297 -- Example: all customers, with their orders if they have any
298 SELECT c.first_name, o.order_id
299 FROM customers c
300 LEFT JOIN orders o ON c.customer_id = o.customer_id;
301
302 -- Example: all orders, including any that somehow lack a valid c
303 -- (shouldn't happen with FK constraints, but useful to check)
304 SELECT c.first_name, o.order_id
305 FROM customers c
306 RIGHT JOIN orders o ON c.customer_id = o.customer_id;
```

100 % 12 0 Ln: 298, Ch: 1 (105 chars, 3 li

Results Messages

	first_name	order_id
1	Aarav	1001
2	Aarav	1004
3	Priya	1002
4	Rohan	1003
5	Rohan	1008
6	Sneha	1005
7	Vikram	1006
8	Ananya	1007
9	Karan	1009
10	Divya	1010

RIGHT JOIN — the mirror of LEFT JOIN. All rows from the right table are kept, left table rows are NULL when there's no match.

-- Example: all orders, including any that somehow lack a valid customer record

-- (shouldn't happen with FK constraints, but useful to check)

SELECT c.first_name, o.order_id

FROM customers c

RIGHT JOIN orders o ON c.customer_id = o.customer_id;

```

302 -- Example: all orders, including any that somehow lack a valid customer record
303 -- (shouldn't happen with FK constraints, but useful to check)
304 SELECT c.first_name, o.order_id
305 FROM   customers c
306 RIGHT JOIN orders o ON c.customer_id = o.customer_id;
307

```

00 % 12 0 ↑ ↓

Ln: 306, Ch: 54 (106 chars, 3 lines) SPC CRLF UTF

Results Messages

first_name	order_id
Aarav	1001
Priya	1002
Rohan	1003
Aarav	1004
Sneha	1005
Vikram	1006
Ananya	1007
Rohan	1008
Karan	1009
0	1010

FULL OUTER JOIN — combines both: all rows from both tables are kept. Rows that don't have a match on either side get NULLs for the unmatched columns.

-- Useful when you want to see ALL customers AND ALL orders,

-- even if some don't have a counterpart

```
SELECT c.first_name, o.order_id
```

```
FROM   customers c
```

```
FULL OUTER JOIN orders o ON c.customer_id = o.customer_id;
```

```
309 -- Useful when you want to see ALL customers AND ALL orders,
310 -- even if some don't have a counterpart
311 SELECT c.first_name, o.order_id
312 FROM customers c
313 FULL OUTER JOIN orders o ON c.customer_id = o.customer_id;
314
315
316 -- customer_id 999 does not exist in the customers table
317 INSERT INTO orders VALUES (1099, 999, '2024-09-01', 'Pending', 500.00);
```

100 % 12 0 Ln: 314, Ch: 1 (113 chars, 3 li

	first_name	order_id
1	Aarav	1001
2	Aarav	1004
3	Priya	1002
4	Rohan	1003
5	Rohan	1008
6	Sneha	1005
7	Vikram	1006
8	Ananya	1007
9	Karan	1009
10	Divya	1010

When to use FULL OUTER JOIN: Data reconciliation scenarios — for example, finding customers with no orders AND orders linked to deleted customers at the same time in one query.

Q23. Foreign Key relationships — what happens with customer_id = 999?

All FK relationships in the schema:

Child Table	FK Column	References
-------------	-----------	------------

orders	customer_id	customers(customer_id)
--------	-------------	------------------------

order_items	order_id	orders(order_id)
-------------	----------	------------------

order_items	product_id	products(product_id)
-------------	------------	----------------------

Test — insert an order for a non-existent customer:

```
-- customer_id 999 does not exist in the customers table
```

```
INSERT INTO orders VALUES (1099, 999, '2024-09-01', 'Pending', 500.00);
```

```
315
316 -- customer_id 999 does not exist in the customers table
317 INSERT INTO orders VALUES (1099, 999, '2024-09-01', 'Pending', 500.00);
318
319 -----
320
321 --Section E - Advanced Concepts (CASE, ACID, Transactions)
322 -- CASE works like an if-else inside a SELECT statement
323 SELECT product_name
```

100 % 12 0 Ln: 317, Ch: 1 (71 chars, 1 lines) SPC CRLF UTI

Messages

Msg 547, Level 16, State 0, Line 317
The INSERT statement conflicted with the FOREIGN KEY constraint "FK__orders__customer__5629CD9C". The conflict occurred because the statement has been terminated.

Completion time: 2026-06-28T13:01:22.5139504+05:30

What happens: SSMS raises:

The INSERT statement conflicted with the FOREIGN KEY constraint "FK__orders__customer...". The statement has been terminated.

The FK constraint enforces **referential integrity** — the database refuses to create a child record that has no parent. Without FKs, we could end up with "orphan" orders that point to customers who don't exist, making JOINS unreliable and reports inaccurate.

Section E — Advanced Concepts (CASE, ACID, Transactions)

Q24. CASE — classify products into price tiers

-- CASE works like an if-else inside a SELECT statement

```
SELECT product_name,  
       unit_price,  
       CASE  
         WHEN unit_price < 1000      THEN 'Budget'  
         WHEN unit_price BETWEEN 1000 AND 3000 THEN 'Mid-Range'  
         WHEN unit_price > 3000      THEN 'Premium'  
       END AS price_tier  
FROM products  
ORDER BY unit_price;
```

```
322 -- CASE works like an if-else inside a SELECT statement
323 SELECT product_name,
324        unit_price,
325        CASE
326            WHEN unit_price < 1000 THEN 'Budget'
327            WHEN unit_price BETWEEN 1000 AND 3000 THEN 'Mid-Range'
328            WHEN unit_price > 3000 THEN 'Premium'
329        END AS price_tier
330 FROM products
331 ORDER BY unit_price;
332
333
334
335
```

00 % 12 0 Ln: 332, Ch: 1 (310 chars, 9 lines)

Results Messages

	product_name	unit_price	price_tier
1	Cushion Covers (Set)	599.00	Budget
2	Cotton T-Shirt	799.00	Budget
3	Laptop Stand	899.00	Budget
4	Bedsheet Set	1299.00	Mid-Range
5	Wireless Earbuds	1499.00	Mid-Range
6	Smart Watch	2999.00	Mid-Range
7	Bluetooth Speaker	3499.00	Premium
8	Running Shoes	4599.00	Premium

Observation:

- Budget: Laptop Stand (₹899), Cushion Covers (₹599)
- Mid-Range: Wireless Earbuds, Cotton T-Shirt, Bedsheet Set, Smart Watch
- Premium: Running Shoes (₹4599), Bluetooth Speaker (₹3499)

CASE is very handy for creating derived columns or bucketing continuous values. The result can also be used in GROUP BY for category-level counts.

Q25. CASE inside aggregate — Delivered vs Not Delivered count

```
-- Using CASE inside SUM() to conditionally count rows
-- SUM(CASE WHEN ... THEN 1 ELSE 0 END) is a classic pivot pattern
SELECT
    SUM(CASE WHEN status = 'Delivered' THEN 1 ELSE 0 END) AS delivered_count,
```

SUM(CASE WHEN status <> 'Delivered' THEN 1 ELSE 0 END) AS not_delivered_count

FROM orders;

```
336 -- SUM(CASE WHEN ... THEN 1 ELSE 0 END) is a classic pivot pattern
337 SELECT
338     SUM(CASE WHEN status = 'Delivered' THEN 1 ELSE 0 END) AS delivered_count,
339     SUM(CASE WHEN status <> 'Delivered' THEN 1 ELSE 0 END) AS not_delivered_count
340 FROM orders;
341
342
343 -- This transaction does 4 things atomically:
344 -- 1. Inserts a new order for customer 102
345 -- 2. Inserts two line items for that order
346 -- 3. Updates stock quantities of both purchased products
347 -- 4. Commits the transaction
```

100 % 12 0 Ln: 341, Ch: 1 (252 chars, 5 lines) SPC CRLF UTF

Results Messages

delivered_count	not_delivered_count
6	4

Observation: 6 Delivered, 4 Not Delivered (2 Shipped + 1 Pending + 1 Cancelled). This pattern is more efficient than running two separate queries with different WHERE clauses because it scans the table only once.

Q26. ACID Properties — explained with a real-world bank transfer example

ACID stands for the four guarantees a reliable database transaction must provide:

A — Atomicity A transaction is all-or-nothing. Either every step completes successfully or none of them apply.

Example: Transferring ₹5000 from Account A to Account B involves two steps — debit A, credit B. If the debit succeeds but the system crashes before the credit, Atomicity ensures the debit is rolled back. The money doesn't vanish.

C — Consistency A transaction takes the database from one valid state to another, always respecting all defined rules and constraints.

Example: A bank's rule is "balance cannot go negative." Consistency ensures that no transaction — even a partial one — can leave an account with a negative balance and commit it.

I — Isolation Concurrent transactions don't see each other's intermediate (uncommitted) data.

Example: If two people both try to book the last seat on a flight at the same time, isolation ensures one transaction completes first and the other sees the updated (no seat available) state — not the old state that had a seat.

D — Durability Once a transaction is committed, it remains committed even if the server crashes immediately after.

Example: After confirming a payment and receiving a "Payment Successful" message, the data is written to disk (or replicated). A power failure won't undo that transaction.

Q27. Full SQL Transaction — new order with stock update and rollback safety

-- This transaction does 4 things atomically:

-- 1. Inserts a new order for customer 102

-- 2. Inserts two line items for that order

-- 3. Updates stock quantities of both purchased products

-- 4. COMMITs if all steps succeed, otherwise ROLLBACKs everything

BEGIN TRY

BEGIN TRANSACTION;

-- Step 1: Insert the new order

INSERT INTO orders (order_id, customer_id, order_date, status, total_amount)

VALUES (1011, 102, CAST(GETDATE() AS DATE), 'Pending', 1598.00);

-- Step 2a: First line item — 1 unit of Laptop Stand (product 207, ₹899)

INSERT INTO order_items (item_id, order_id, product_id, quantity, unit_price, discount_pct)

VALUES (5016, 1011, 207, 1, 899.00, 0);

-- Step 2b: Second line item — 1 unit of Cushion Covers (product 208, ₹599)

```
INSERT INTO order_items (item_id, order_id, product_id, quantity, unit_price, discount_pct)
VALUES (5017, 1011, 208, 1, 599.00, 0);
```

```
-- Step 3a: Reduce stock for Laptop Stand by 1
```

```
UPDATE products
```

```
SET stock_qty = stock_qty - 1
```

```
WHERE product_id = 207;
```

```
-- Step 3b: Reduce stock for Cushion Covers by 1
```

```
UPDATE products
```

```
SET stock_qty = stock_qty - 1
```

```
WHERE product_id = 208;
```

```
-- If we reached here without errors, commit everything
```

```
COMMIT TRANSACTION;
```

```
PRINT 'Transaction committed successfully. Order 1011 placed.';
```

```
END TRY
```

```
BEGIN CATCH
```

```
-- If anything above failed, undo all changes
```

```
ROLLBACK TRANSACTION;
```

```
PRINT 'Transaction rolled back due to error: ' + ERROR_MESSAGE();
```

```
END CATCH;
```

```
GO
```

-- Verify the new order was inserted

```
SELECT * FROM orders WHERE order_id = 1011;
```

-- Verify stock was reduced for both products

```
SELECT product_id, product_name, stock_qty
```

```
FROM products
```

```
WHERE product_id IN (207, 208);
```

The screenshot shows a SQL IDE with a script editor and a results pane. The script editor contains a transaction that inserts a new order and updates stock quantities. The results pane shows the output of the script, including the new order and the updated stock levels for products 207 and 208.

```
342
343 -- This transaction does 4 things atomically:
344 -- 1. Inserts a new order for customer 102
345 -- 2. Inserts two line items for that order
346 -- 3. Updates stock quantities of both purchased products
347 -- 4. COMMITs if all steps succeed, otherwise ROLLBACKs everything
348
349 BEGIN TRY
350     BEGIN TRANSACTION;
351
352     -- Step 1: Insert the new order
353     INSERT INTO orders (order_id, customer_id, order_date, status, total_amount)
354     VALUES (1011, 102, CAST(GETDATE() AS DATE), 'Pending', 1598.00);
355
356     -- Step 2a: First line item - 1 unit of Laptop Stand (product 207, ₹899)
357     INSERT INTO order_items (item_id, order_id, product_id, quantity, unit_price
358     VALUES (5016, 1011, 207, 1, 899.00, 0);
```

100 % 12 0 Ln: 343, Ch: 1 (1799 chars, 50 lines) SPC CRLF UTF-8

Results Messages

	order_id	customer_id	order_date	status	total_amount
1	1011	102	2026-06-28	Pending	1598.00

	product_id	product_name	stock_qty
1	207	Laptop Stand	179
2	208	Cushion Covers (Set)	399

Observation: After running this, order 1011 appears in orders and the stock for product 207 (Laptop Stand) goes from 180 to 179, and product 208 (Cushion Covers) from 400 to 399. If I intentionally broke

step 3 (e.g., set stock to a negative value to trigger the CHECK constraint), the CATCH block fires and the entire transaction — including the new order rows — gets rolled back cleanly. This is Atomicity in action.

Summary of Key Learnings

Working through this task on SSMS gave me hands-on exposure to building a relational database from scratch and querying it in progressively complex ways. A few things that stood out:

- Constraints (PRIMARY KEY, UNIQUE, CHECK, FOREIGN KEY) are not optional extras — they are the safety net that keeps data trustworthy at the database level.
- Indexes dramatically change query performance but they come with a write-overhead trade-off. Choosing which columns to index requires understanding how the data will be queried.
- SARGable queries (avoiding functions on indexed columns in WHERE clauses) is something I hadn't thought about deeply before — Q12 made that very clear.
- The CASE inside SUM() pattern for conditional aggregation is something I'll use often for quick pivot-style summaries.
- Transactions with TRY/CATCH/ROLLBACK are essential for any multi-step operation where partial completion would leave the database in an inconsistent state.